LeetCode 290: Word Pattern

1. Problem Title & Link

Title: LeetCode 290: Word Pattern

Link: https://leetcode.com/problems/word-pattern/

2. Problem Statement (Short Summary)

Given:

- A pattern string pattern
- A sentence string s

Determine whether s follows the same pattern.

A valid pattern means:

One character ↔ One word

and

One word ↔ One character

This is called a **one-to-one mapping (bijection)**.

Return:

true

if the pattern matches the sentence, otherwise:

false

3. Examples (Input → Output)

Example 1

Input:

pattern = "abba"

s = "dog cat cat dog"

Output:

true

Mapping:

a → dog

b → cat

Pattern:

a b b a

dog cat cat dog

Matches.

Example 2

Input:

pattern = "abba"

s = "dog cat cat fish"

Output:

false

Last character:

a → dog

but word is:

fish

Mismatch.

Example 3

Input:

pattern = "aaaa"

s = "dog cat cat dog"

Output:

false

Because:

a

cannot map to multiple words.

Example 4

Input:

pattern = "abba"

s = "dog dog dog dog"

Output:

false

Because:

a and b

cannot map to the same word.

4. Constraints

1 <= pattern.length <= 300

1 <= s.length <= 3000

Important:

Need one-to-one mapping.

Not just one-direction mapping.

5. Core Concept (Pattern / Topic)

HashMap

Secondary Topics:

- String Processing
- Bijective Mapping

6. Thought Process (Step-by-Step Explanation)

Understanding the Problem

Input:

pattern = "abba"

s = "dog cat cat dog"

Pairs:

a → dog

b → cat

b → cat

a → dog

Consistent.

Answer:

true

What Can Go Wrong?

Case 1

Input:

pattern = "abba"

s = "dog cat cat fish"

Mapping:
a → dog

b → cat

b → cat

a → fish
Now:
a
maps to:
dog and fish
Not allowed.

Case 2
Input:
pattern = "abba"

s = "dog dog dog dog"
Mapping:
a → dog

b → dog
Now:
Two pattern characters
map to:
same word
Not allowed.

Key Insight
Need to enforce BOTH:
Character → Word
and
Word → Character
Therefore:
Need Two HashMaps
or

HashMap + HashSet

7. Visual / Intuition Diagram (ASCII Diagram)

```
Example:
abba

dog cat cat dog
Mapping:
a → dog

b → cat
Check:
dog → a

cat → b
Both directions valid.

Invalid Example
abba

dog dog dog dog
a → dog

b → dog
Two characters:
a

b
map to same word.
Invalid.
```

8. Pseudocode (Language Independent)

```
split sentence into words

if lengths differ

    return false

create charToWord map

create wordToChar map

for each position

    check both mappings

    if conflict
```

```
         return false

return true
```

9. Code Implementation

Python

```
class Solution:
    def wordPattern(self, pattern: str, s: str) -> bool:

        words = s.split()

        if len(pattern) != len(words):
            return False

        char_to_word = {}
        word_to_char = {}

        for ch, word in zip(pattern, words):

            if ch in char_to_word:

                if char_to_word[ch] != word:
                    return False

            else:
                char_to_word[ch] = word

            if word in word_to_char:

                if word_to_char[word] != ch:
                    return False

            else:
                word_to_char[word] = ch

        return True
```

Java

```
import java.util.*;

class Solution {

    public boolean wordPattern(String pattern, String s) {

        String[] words = s.split(" ");

        if(pattern.length() != words.length)
```

```
            return false;

        HashMap<Character,String> charToWord =
                new HashMap<>();

        HashMap<String,Character> wordToChar =
                new HashMap<>();

        for(int i = 0; i < pattern.length(); i++) {

            char ch = pattern.charAt(i);
            String word = words[i];

            if(charToWord.containsKey(ch)) {

                if(!charToWord.get(ch).equals(word))
                    return false;

            } else {

                charToWord.put(ch, word);
            }

            if(wordToChar.containsKey(word)) {

                if(wordToChar.get(word) != ch)
                    return false;

            } else {

                wordToChar.put(word, ch);
            }
        }

        return true;
    }
}
```

10. Time & Space Complexity

Time Complexity

O(n)

where:

n = number of words

Space Complexity

O(n)

For storing mappings.

11. Common Mistakes / Edge Cases

Mistake 1

Checking only:

Character → Word

mapping.

Fails:

abba

dog dog dog dog

Mistake 2

Not checking length.

Example:

pattern = "ab"

s = "dog cat fish"

Lengths differ.

Immediately:

false

Mistake 3

Using one HashMap only.

Need:

Two-way validation.

Edge Cases

Single Character

Input:

a

dog

Output:

true

Length Mismatch

Input:

ab

dog
Output:
false

12. Detailed Dry Run

Input:
pattern = "abba"

s = "dog cat cat dog"
Words:
["dog","cat","cat","dog"]

Pattern	Word	Action
a	dog	Store
b	cat	Store
b	cat	Valid
a	dog	Valid

No conflicts.
Return:
true

13. Common Use Cases (Real-Life / Interview)

Dictionary Encoding

Mapping symbols to words.

Language Processing

Grammar pattern validation.

Data Transformation

Verifying consistent mappings.

Interview Purpose

Introduces:
HashMap
Bidirectional Mapping

14. Common Traps (Important!)

Trap 1

Assuming one map is enough.

It isn't.

Need:

Character → Word

Word → Character

Trap 2

Ignoring duplicate words.

Example:

a → dog

b → dog

Invalid.

Trap 3

Not checking word count equals pattern length.

15. Builds To (Related LeetCode Problems)

LC 290 - Word Pattern

↓

LC 205 - Isomorphic Strings

↓

LC 49 - Group Anagrams

↓

LC 242 - Valid Anagram

↓

LC 981 - Time Based Key Value Store

These strengthen HashMap mastery.

16. Alternate Approaches + Comparison

Approach	Time	Space	Interview Rating
Nested Search	$O(n^2)$	O(1)	Poor

Single Map	Incorrect	-	Wrong
Two HashMaps	O(n)	O(n)	Best

Approach 1: Nested Validation

Check every pair manually.

Complex and inefficient.

Approach 2: Two HashMaps (Chosen)

Most common interview solution.

Approach 3: Index Pattern Trick

Advanced solution:

list(map(pattern.index, pattern))

==

list(map(words.index, words))

Elegant but less interview-friendly.

17. Why This Solution Works (Short Intuition)

A valid pattern requires a **bijection**:

One character ↔ One word

Using two HashMaps guarantees consistency in both directions and immediately detects any conflict.

18. Variations / Follow-Up Questions

Follow-Up 1

Check pattern between two strings.

Related:

LC 205

Isomorphic Strings

Follow-Up 2

Allow one character to map to multiple words.

How would the solution change?

Follow-Up 3

Return the actual mapping.

Example:

{
a : dog,
b : cat
}

Follow-Up 4

Count the number of unique valid mappings.

Interview Takeaway

This is one of the most important **HashMap Mapping Problems**.

The core lesson is:

One-to-One Mapping

Whenever you hear:

Pattern Matching

Symbol Mapping

Character Mapping

Relationship Validation

think:

Two HashMaps

Bidirectional Validation

This same idea appears in Isomorphic Strings, Anagrams, Encoders, Decoders, and many medium-level interview questions.